

System Role: You are a highly critical, world-renowned AI Scientist and Peer Reviewer specializing in the evaluation of Large Language Models (LLMs) for ****Precision Question Answering Agriculture****.

Task: Read the paper provided below. Based on the paper's stated goals, its chosen dataset, and its methodology, identify one major weakness or missed opportunity in its experimental design. Then, propose a rigorous alternative or addition that would strengthen the paper's claims.

****Input Text (Full Paper):****

2. Materials and Methods

2.1. Algorithm Introduction

The algorithmic principle of Sem-RAG is shown in Algorithm 1, and its overall process

is further illustrated in Figure 1. The algorithm is divided into two stages, including the

construction of a knowledge base and the retrieval inference stage.

In the stage of building the knowledge base, firstly, the professional document T of corn

planting knowledge is divided into fixed-length chunks to form $\{C_1, C_2, \dots, C_n\}$; then, each

chunk C_i in $\{C_1, C_2, \dots, C_n\}$ is processed sequentially, semantically related triple information

Q_i is extracted from C_i , and a graph G_i is established based on Q_i . Next, we use the Leiden

algorithm to stratify the graph G_i into community lists $\{G_{i1}, G_{i2}, \dots, G_{ip}\}$ and generate

a themed community summary S_{ij} for each community G_{ij} , which includes <topic, key relationship, condition, step>; then, we vectorize the original text of each chunk C_i to

obtain V_i and vectorize the thematic summary S_{ij} in each chunk C_i to obtain U_{ij} ; finally,

based on V_i and U_{ij} , we construct the surface semantic vector knowledge base $VS_{shallow}$

and the fine-grained semantic vector knowledge base VS_{fine} for retrieval, respectively.

Algorithm 1 Semantic retrieval-augmented generation.

Require: Input text T (corn-growing docs) and question P

Ensure: Retrieved answers A

1: Parse T into fixed-length chunks $\{C_1, C_2, \dots, C_n\}$

2: for each chunk C_i in $\{C_1, C_2, \dots, C_n\}$ do

3: Extract semantic triples Q_i from C_i , where $Q_i = \{(\text{entity}_1, \text{entity}_2, \text{relation})\}$

4: Build a per-chunk graph G_i from Q_i

5: Run the Leiden algorithm on G_i to detect communities $\{G_{i1}, G_{i2}, \dots, G_{ip}\}$

6: for each community G_{ij} in $\{G_{i1}, \dots, G_{ip}\}$ do

7: Summarize G_{ij} into a thematic summary S_{ij} (topic, key relations, conditions, steps)

8: end for

9: Vectorize C_i (original text) with a shallow encoder $\rightarrow V_i$

10: Vectorize each S_{ij} with a fine-grained encoder $\rightarrow U_{ij}$

11: end for

12: Build two vector stores: $VS_{shallow} = \{(ID(C_i), V_i)\}$ $VS_{fine} = \{((ID(C_i), ID(G_{ij})), U_{ij})\}$

13: Fine-tune the LLM on a small set of domain-specific Q&A pairs

14: Encode P with the shallow encoder $\rightarrow q_s$

15: Calculate similarity between q_s and all V_i in $VS_{shallow}$; sort and select top- k chunks

$\{Z_1, Z_2, \dots, Z_k\}$ with IDs $\{ID(Z_1), \dots, ID(Z_k)\}$

16: Collect fine summaries linked to the retrieved chunks: $U^* = \{S$

$i, j \mid ID(C_i) \in$

$\{ID(Z_1), \dots, ID(Z_k)\}\}$

17: Rank U^* by similarity to q_s and select top- r summaries S^*

18: Provide both the retrieved chunk texts $\{Z_1, \dots, Z_k\}$ and selected summaries S^* to the LLM

19: LLM generates answers A based on $\{Z_1, \dots, Z_k\}$ and S^*

20: Output answers A

Query Encode

Similarity

search

User Response

Embedding

Model

Index

Question: Is corn in Xinjiang sown in the spring planting area or the summer planting area?

Answer: The northern part of Xinjiang belongs to the spring-sown corn area, while the southern part is a mixed area for both spring and summer sowing.

Entity extraction C_i

Constructing Q_i with triples

Build the graph G_i

Lora fine-tuning

Community partitioning

In the retrieval reasoning stage, firstly, LoRA fine-tuning work was carried out on

specific domain data for the large language model to enhance its ability to generate replies

after retrieval; secondly, the user query vector P was encoded as q_s , and we performed

similar chunk queries with all V_{is} on the surface semantic knowledge base $V_{Sshallow}$ to

obtain a list of similar chunks $\{Z_1, Z_2, \dots, Z_k\}$; then, the candidate graph community

summary information in the fine-grained semantic vector knowledge base V_{Sfine} is located

through the similar chunk list $\{Z_1, Z_2, \dots, Z_k\}$. The user query vector q_s is searched

for similarity with the candidate graph community summary information to obtain the

most relevant r graph community summaries, forming a set of graph summaries S^* .

Finally, the similar chunk information $\{Z_1, Z_2, \dots, Z_k\}$ and the graph community summary

information S^* are used as semantic information references for the large language model's

response to generate query results.

2.2. LoRA Fine-Tuning

LoRA (Low-Rank Adaptation) is a parameter-efficient fine-tuning method [27-29], whose core idea is to introduce trainable "adapters" into the weight matrix of the pretrained

model through low-rank decomposition. Based on freezing the pre-trained model parameters, it achieves efficient adaptation to reduce training and storage costs. In this

study, we enhanced the adaptation ability of the large language model to domain knowledge

using the LoRA method based on a small number of samples in the field of maize knowledge question-answering. At the same time, the LoRA method was used at the task level to evaluate the extraction capability of graph nodes to align the results of the

extraction evaluation.

As shown in Figure 2, the algorithm principle of the LoRA fine-tuning mechanism is to first select the layers that need to be adapted in the large-scale pre-trained model, that is, the attention projection matrix in the Transformer structure. Secondly, in the forward propagation stage, the input vector x is calculated using the weight modification module to obtain the output y . The calculation method of the weight modification module is shown in Equation (1). Among them, the original weight matrix of the attention projection matrix is $W \in \mathbb{R}^{d \times k}$. The LoRA fine-tuning mechanism does not directly update the original weight matrix but introduces two low-rank matrices $A \in \mathbb{R}^{d \times r}$ and $B \in \mathbb{R}^{r \times k}$ and constructs a low-rank update term ΔW through the low-rank update process. Wx is the frozen output of the pre-training part, and $(AB)x$ is the trainable correction provided by the LoRA mechanism. The calculation method for the low-rank update process is shown in Equation (2), where α is the scaling factor used to control the stability of the update amplitude. Then, in the backpropagation stage, only the gradients of low-rank matrices A and B are calculated and updated, while the pre-training parameter W remains unchanged to reduce the number of parameters that need to be updated and stored during the training process; finally, at the end of the training, we save the updated weights ΔW . In the model inference stage, the weight matrix W of the pre-trained model is superimposed with the low-rank updated weights ΔW to obtain the merged weight matrix W' , and model inference is performed based on W' .

The advantages of LoRA include training a very small number of newly added parameters to achieve near full fine-tuning effects; the LoRA weights of different tasks being loaded as modules without affecting the main model; and avoiding catastrophic forgetting.

Overall, the core idea of LoRA is to compress the adaptability of large models into a small number of parameter updates through low-rank approximation, thereby achieving efficient transfer learning.

$$y = (W + \Delta W)x \quad (1)$$

$$\Delta W =$$

$$\alpha$$

$$r$$

$$AB$$

2.3. Contextual Semantic Association Processing Module

During the process of knowledge base construction, text chunks often contain complex semantic relationships, and it is difficult for direct storage to support efficient semantic retrieval and reasoning. Therefore, this study designs a contextual semantic association processing module, aiming to transform natural language knowledge into structured,

thematic, and vectorizable multi-level representations. This module mainly includes steps such as semantic triple information extraction, community division and summary generation, and semantic vectorization.

2.3.1. Triple Extraction and Knowledge Graph Construction

For each text chunk C_i , this study utilizes the LLM's general extraction capability to extract triple information. Unlike traditional dependency syntax analysis and rulebased relationship extraction methods, LLM has stronger generalization and contextual understanding abilities in complex contexts and cross-domain scenarios, thus enabling more accurate recognition of semantic structures.

$$Q_i = \{(h, r, t) | h, t \in V, r \in R\} \quad (3)$$

The extracted triples are in the form shown in Equation (3). Here, h represents the head entity, t represents the tail entity, and r represents the semantic relationship. Based on the triple set Q_i , a local knowledge graph can be constructed, and its representation form is as shown in Equation (4). Here, V is the entity set, and E is the set of relationship edges.

The representation form of E is as shown in Equation (5).

$$G_i = (V, E)$$

$$E = \{(h, r, t) | (h, r, t) \in Q_i\}$$

2.3.2. Community Partitioning

As the number of triples increases, the scale and complexity of the knowledge graph

G_i grow rapidly, necessitating further structural decomposition. This study employs the

Leiden community detection algorithm to partition the graph, with the maximization of

modularity Q as the optimization objective. The calculation of Q is shown in Equation (6).

Here, A_{ij} represents the edge weight between nodes i and j , and k_i denotes the degree of node i , which is calculated as shown in Equation (7). m is the total weight of the edges in the graph, and δ is the indicator function, taking a value of 1 when nodes i and j belong to the same community.

$$Q =$$

$$\frac{1}{2m} \sum_{i,j} A_{ij} - \frac{k_i k_j}{2m}$$

$$\sum_{i,j} \delta(c_i, c_j) \quad (6)$$

$$k_i = \sum_j A_{ij} \quad (7)$$

$$\delta(c_i, c_j)$$

$$A_{ij} - \frac{k_i k_j}{2m}$$

$$\sum_{i,j} \delta(c_i, c_j) \quad (6)$$

$$k_i = \sum_j A_{ij} \quad (7)$$

$$\delta(c_i, c_j)$$

$$A_{ij} - \frac{k_i k_j}{2m}$$

$$\sum_{i,j} \delta(c_i, c_j) \quad (6)$$

$$k_i = \sum_j A_{ij} \quad (7)$$

The Leiden algorithm introduces three-stage iterative optimization based on the Louvain

framework: firstly, during the node movement phase, the community membership of nodes is adjusted to improve local modularity while ensuring community connectivity;

secondly, in the community refinement stage, the initial community should be split

and optimized to avoid the emergence of disconnected or inferior communities; finally,

in the graph aggregation stage, the community is compressed into supernodes, a

new graph is constructed, and iterations are performed again until convergence. After Leiden algorithm's community partitioning, a community set with high semantic consistency is ultimately formed.

2.3.3. Community Summary Construction and Semantic Vectorization

In order to improve the readability and retrievability of knowledge within the community, research is conducted on generating thematic semantic summaries S_{ij} for each community G_{ij} . The construction process is as follows: firstly, the keyword extraction process is carried out, and the TextRank algorithm is used to identify words with strong thematic relevance; secondly, relationship filtering is performed, the triples within the community are sorted based on relationship weights, and core knowledge is retained; finally, condition and step extraction is performed, combined with use of the semantic parsing ability of the large language model, to identify conditional patterns and operational processes. The summary S_{ij} formed through this process can be formally represented as (theme, key relationships, conditions, steps). This structured representation compresses complex graph knowledge into interpretable knowledge units, providing support for subsequent fine-grained retrieval.

After obtaining the original chunks and community summaries, they need to be mapped to a vector space to support semantic retrieval. By embedding the thematic community summary S_{ij} through an encoding model, a fine-grained semantic vector U_{ij} is obtained, and finally, a fine-grained semantic knowledge base is constructed.

The representation of U_{ij} is shown in Equation (8), where f_{θ} represents the encoder function.

The representation of VS_{fine} is shown in Equation (9).

$$U_{ij} = f_{\theta}(S_{ij}) \quad (8)$$

$$VS_{fine} = \{U_{11}, U_{12}, \dots, U_{np}\}$$

****Required Output: Critical Experimental Design setup ****

Please structure your response into the following three parts:

1. ****Summary of the Paper's Core Method & Evaluation:**** Briefly state the system and the primary evaluation metrics used.

2. ****Identified Flaw/Missed Opportunity in Experimental Design:**** State ****one specific area**** where the paper's experiment could be significantly improved. This must relate to baselines, datasets, or evaluation metrics.

* ***Example areas:** * Missing a necessary domain-specific baseline, using generic metrics when more precise factual consistency checks are available, or failing to test generalization across different agricultural regions.

3. ****Proposed Alternative/Addition (The "AI Scientist's Better Way"):** Detail the specific experimental element that should have been used instead, or in addition.

* ****If a Baseline is Missing:**** Which specific model (name and type) should have been included?

* ****If a Metric is Weak:**** Which specific, state-of-the-art metric (e.g., RAGAS, LLM-as-a-Judge, FEQA) should have been used, and why is it superior in this agricultural domain?

* **If a Significance Test is Missing:** Which statistical test (e.g., T-test, ANOVA) should have been run to prove the results were statistically robust?